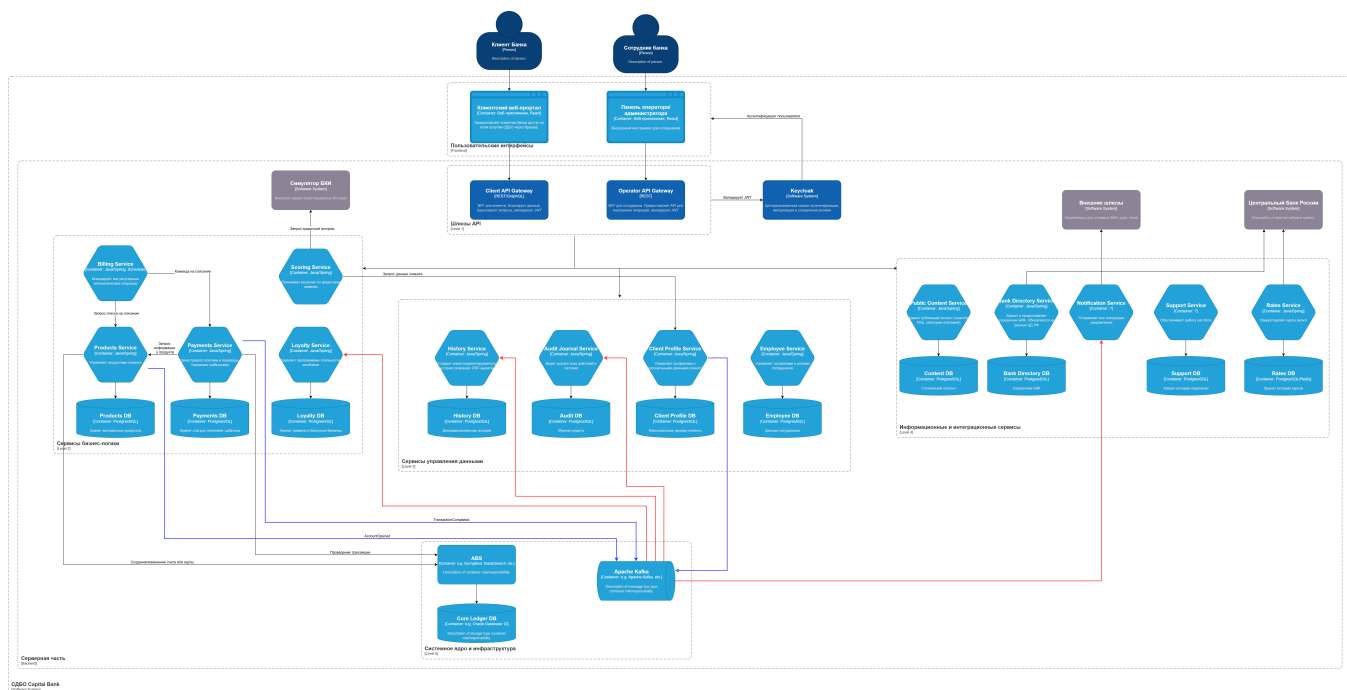


Discovery: Архитектура проекта

- Архитектурная диаграмма
- Каталог микросервисов Capital Bank
 - 1. Client API Gateway
 - 2. Operator API Gateway
 - 3. Симулятор ABC
 - 4. Products Service
 - 5. Payments Service
 - 6. Billing Service
 - 7. Scoring Service
 - 8. Loyalty Service
 - 9. Client Profile Service
 - 10. Employee Management Service
 - 11. History Service
 - 12. Audit Journal Service
 - 13. Rates Service
 - 14. Public Content Service
 - 15. Notification Service
 - 16. Support Service

Архитектурная диаграмма

V1



V2

- **Ключевые обязанности:**
 - Агрегация данных: Собирает данные из нескольких внутренних микросервисов и отдает их клиенту одним запросом, снижая количество сетевых вызовов.
 - Адаптация API: Предоставляет фронтенду удобный и гибкий GraphQL API, "под капотом" транслируя его в REST-вызовы к внутренним сервисам.
 - Аутентификация и безопасность: Валидирует JWT-токен каждого входящего запроса, взаимодействуя с Keycloak.
 - Маршрутизация: Направляет запросы к соответствующим внутренним микросервисам.
 - Инкапсуляция: Скрывает сложность внутренней микросервисной архитектуры от клиентского приложения.
- **Взаимодействие с другими сервисами:**
 - Синхронные вызовы (GraphQL/REST):
 - -> Вызывает (Calls): Практически все сервисы уровней 2, 3 и 4 для получения данных (Products Service, Client Profile Service, History Service, Rates Service и т.д.).
 - <- Вызывается (Is Called By): Клиентский Веб-Портал.
 - -> Вызывает (Calls): Keycloak (для получения публичных ключей для валидации токенов).
 - Асинхронные взаимодействия (Kafka): Не участвует.
- **База данных и хранимые сущности:**
 - Отсутствует. Этот сервис является stateless (не хранит состояние). Вся конфигурация маршрутов хранится в его конфигурационных файлах.
- **Ключевые эндпоинты API:**
 - POST /graphql: Единый эндпоинт для всех GraphQL-запросов (Query и Mutation). Спецификация API определяется его GraphQL-схемой.

2. Operator API Gateway

- **Назначение:**

Служит единой, защищенной точкой входа (Backend-for-Frontend) для внутренних инструментов, используемых сотрудниками банка (в нашем случае — для "Панели Оператора").
- **Ключевые обязанности:**
 - Предоставление API для администрирования: Открывает эндпоинты для выполнения служебных и административных задач (управление клиентами, продуктами, просмотр журнала аудита).
 - Аутентификация и Авторизация: Валидирует JWT-токен сотрудника и может применять дополнительную проверку ролей (ADMIN, OPERATOR) перед маршрутизацией.
 - Маршрутизация: Направляет запросы к соответствующим внутренним микросервисам.
- **Взаимодействие с другими сервисами:**
 - Синхронные вызовы (REST):
 - -> Вызывает (Calls): Сервисы, предоставляющие административные функции (Client Profile Service, Employee Management Service, Audit Journal Service, Products Service и т.д.).
 - <- Вызывается (Is Called By): Панель Оператора.
 - -> Вызывает (Calls): Keycloak (для валидации токенов).
 - Асинхронные взаимодействия (Kafka): Может публиковать события о действиях операторов в Kafka, которые затем слушает Audit Journal Service.
- **База данных и хранимые сущности:**
 - Отсутствует. Этот сервис также является stateless.
- **Ключевые эндпоинты API:**
 - GET /admin/clients: Получить список клиентов.
 - POST /admin/clients/{id}/block: Заблокировать клиента.
 - GET /admin/audit?userId={id}: Получить историю действий по пользователю.

3. Симулятор АБС

- **Назначение:**

Моделирует монолитное ядро банка (Core Banking System). Является единственным источником правды по деньгам и отвечает за целостность финансовых данных.
- **Ключевые обязанности:**
 - Управление счетами на финансовом уровне: Создание записей о счетах, хранение их балансов.
 - Выполнение атомарных транзакций: Гарантирует, что операция "списание-зачисление" либо выполняется полностью, либо не выполняется вообще (принцип ACID).
 - Раздельный учет по картам: Поддерживает логику для кредитных продуктов, ведя отдельный учет операций по каждой конкретной карте.
 - Предоставление простого внутреннего API: Открывает строго ограниченный набор эндпоинтов для выполнения низкоуровневых финансовых операций.
- **Взаимодействие с другими сервисами:**

- Синхронные вызовы (REST):
 - -> Вызывает (Calls): Никого. Является терминальным сервисом, который только исполняет команды.
 - <- Вызывается (Is Called By): Исключительно Products Service (для создания счетов/карт) и Payments Service (для проведения транзакций).
- Асинхронные взаимодействия (Kafka): Не участвует. Не публикует и не слушает события, чтобы обеспечить свою изоляцию и простоту.
- **База данных и хранимые сущности:**
 - Технология: PostgreSQL.
 - Ключевые таблицы:
 - ACCOUNTS: Главная книга счетов. Хранит id, balance, currency_code.
 - CARDS_ABS: Реестр карт как финансовых инструментов. Хранит id и account_id, к которому привязана карта.
 - CARD_LEDGER: Журнал операций по конкретным картам. Позволяет считать траты (spending) по кредитной карте.
 - TRANSACTIONS: Неизменяемый лог всех успешных финансовых проводок (транзакций).
- **Ключевые эндпоинты API:**
 - POST /accounts: Создать новый финансовый счет.
 - GET /accounts/{id}/balance: Получить баланс счета.
 - POST /cards: Создать сущность "карта", привязанную к счету.
 - GET /cards/{id}/spending: Получить сумму трат по конкретной карте за период.
 - POST /card-transactions: Провести транзакцию по карте (с записью в CARD_LEDGER).

4. Products Service

- **Назначение:**
Является центральным сервисом для управления жизненным циклом всех банковских продуктов, принадлежащих клиенту. Он выступает в роли слоя метаданных поверх "финансовых кошельков" в АБС.
- **Ключевые обязанности:**
 - Управление Продуктами: Обработывает создание, закрытие и изменение статуса продуктов (карт, вкладов, кредитов).
 - Хранение Метаданных: Хранит всю нефинансовую информацию о продуктах: псевдонимы, процентные ставки, кредитные лимиты, сроки, тарифы на обслуживание.
 - Оркестрация АБС: Отдает команды Симулятору АБС на создание/изменение счетов и карт в ядре.
 - Проверка Бизнес-Правил: Проверяет возможность выполнения операций с продуктом (например, "можно ли досрочно закрыть этот вклад?").
 - Предоставление Информации о Продуктах: Отдает полную информацию о продуктах клиента, обогащая ее актуальным балансом из АБС.
- **Взаимодействие с другими сервисами:**
 - Синхронные вызовы (REST):
 - -> Вызывает (Calls):
 - Симулятор АБС: для создания финансовых сущностей и получения балансов.
 - Client Profile Service: для получения abs_client_id при создании нового продукта.
 - <- Вызывается (Is Called By):
 - Client API Gateway: для отображения списка продуктов.
 - Operator API Gateway: для управления продуктами клиентов.
 - Payments Service: для получения информации о продукте перед платежом.
 - Billing Service: для получения списка продуктов для регулярных списаний.
 - Асинхронные взаимодействия (Kafka):
 - -> Публикует (Publishes): События о жизненном цикле продуктов, например, AccountOpened, CardBlocked, CreditApplicationCreated.
- **База данных и хранимые сущности:**
 - Технология: PostgreSQL.
 - Ключевые таблицы:
 - PRODUCT_TEMPLATES: Шаблоны всех доступных в банке продуктов (матрицы, которые вы предоставляли).
 - ACCOUNTS: "Зеркало" счетов из АБС, содержит id, abs_account_id, user_id.
 - PRODUCTS: Общая таблица экземпляров продуктов клиента, содержит id, account_id (ссылка на "зеркало"), product_type.
 - CARDS, DEPOSITS, CREDITS: Специализированные таблицы с уникальными атрибутами для каждого типа продукта.
- **Ключевые эндпоинты API:**
 - GET /products: Получить все продукты текущего пользователя.
 - POST /products: Создать новый продукт для пользователя (например, открыть счет).
 - GET /products/{id}: Получить детальную информацию о конкретном продукте (с балансом).
 - PATCH /products/{id}: Изменить метаданные продукта (например, псевдоним).

5. Payments Service

- **Назначение:**
Оркестрирует все платежи и переводы, инициированные пользователем. Является центральным процессором для всех "разовых" денежных движений.
- **Ключевые обязанности:**

- Оркестрация Платежей: Выполняет всю последовательность шагов для проведения перевода: валидация, проверка баланса, вызов АБС, вызов внешних систем (например, СБП).
- Управление Шаблонами: Хранит и управляет пользовательскими шаблонами платежей и автоплатежами.
- Применение Бизнес-Правил в Момент Транзакции: Запрашивает у Products Service лимиты по карте, у Rates Service — актуальный курс, и применяет их.
- Идемпотентность: Обеспечивает защиту от двойного списания с помощью ключей идемпотентности.
- **Взаимодействие с другими сервисами:**
 - Синхронные вызовы (REST):
 - -> Вызывает (Calls):
 - Симулятор АБС: для выполнения финансовых транзакций.
 - Products Service: для получения лимитов и ID счетов перед операцией.
 - Rates Service: для получения актуального курса при конвертации.
 - BankDirectoryService: для валидации БИК.
 - <- Вызывается (Is Called By):
 - Client API Gateway: для инициации перевода.
 - Billing Service: для выполнения служебных списаний.
 - Асинхронные взаимодействия (Kafka):
 - -> Публикует (Publishes): События о статусе транзакций: TransactionCreated, TransactionCompleted, TransactionFailed.
- **База данных и хранимые сущности:**
 - Технология: PostgreSQL.
 - Ключевые таблицы:
 - PAYMENTS: Журнал всех платежных операций и их статусов (PROCESSING, COMPLETED и т.д.).
 - PAYMENT_TEMPLATES: Сохраненные пользователями шаблоны для быстрых платежей.
 - AUTO_PAYMENTS: Настройки автоплатежей, созданных пользователями.
- **Ключевые эндпоинты API:**
 - POST /transfers: Создать новый перевод (основной эндпоинт).
 - GET /payments/{id}/status: Проверить статус платежа.
 - GET /templates, POST /templates: CRUD-операции для управления шаблонами.

6. Billing Service

- **Назначение:**

Отвечает за все регулярные, автоматические и не-пользовательские финансовые операции, которые происходят по расписанию.
- **Ключевые обязанности:**
 - Инициация по Расписанию: Встроенный планировщик (Scheduler) запускает задачи в определенное время (например, раз в день/месяц).
 - Начисление Процентов: Иницирует начисление процентов по вкладам и кредитам.
 - Списание Комиссий: Иницирует списание регулярных плат за обслуживание карт, счетов, уведомлений.
 - Применение Штрафов: Рассчитывает и иницирует списание штрафов за просрочку по кредитам.
- **Взаимодействие с другими сервисами:**
 - Синхронные вызовы (REST):
 - -> Вызывает (Calls):
 - Products Service: для получения списка продуктов, требующих биллинга ("дай все кредиты с просрочкой", "дай все карты для списания обслуживания").
 - Payments Service: для исполнения рассчитанных списаний/начислений. Billing Service не двигает деньги сам, он делегирует это Payments Service.
 - <- Вызывается (Is Called By): Никем. Работает только по внутреннему расписанию.
 - Асинхронные взаимодействия (Kafka): Не участвует.
- **База данных и хранимые сущности:**
 - Может не иметь собственной БД. Вся необходимая информация для работы получается из других сервисов. Логи о запусках задач пишутся в общую систему логирования.
- **Ключевые эндпоинты API:**
 - Может не иметь публичного API, кроме, возможно, эндпоинтов для ручного запуска задач администратором (POST /jobs/run-billing).

7. Scoring Service

- **Назначение:**

Выполняет роль "системы принятия решений" по кредитным заявкам.
- **Ключевые обязанности:**
 - Оценка Заемщика: Анализирует анкетные данные клиента.
 - Интеграция с БКИ: Делает запросы во внешнюю систему (симулятор Бюро Кредитных Историй) для получения кредитного рейтинга.

- Применение Скоринговой Модели: На основе всех данных и внутренних правил выносит вердикт: APPROVED, REJECTED или NEEDS_MANUAL_REVIEW.
- Расчет Персональных Условий: Может рассчитывать индивидуальную процентную ставку и кредитный лимит для одобренной заявки.
- **Взаимодействие с другими сервисами:**
 - Синхронные вызовы (REST):
 - -> Вызывает (Calls):
 - Client Profile Service: для получения полных данных о клиенте.
 - Симулятор БКИ: для получения кредитной истории.
 - <- Вызывается (Is Called By):
 - Products Service: когда создается новая заявка на кредит.
 - Operator API Gateway: для просмотра статуса заявки или ручного запуска скоринга.
 - Асинхронные взаимодействия (Kafka):
 - -> Публикует (Publishes): События о результате скоринга, например, CreditApplicationApproved, CreditApplicationRejected.
- **База данных и хранимые сущности:**
 - Технология: PostgreSQL.
 - Ключевые таблицы:
 - SCORING_APPLICATIONS: Хранит копию данных по заявке и итоговый результат скоринга, включая все промежуточные параметры и вердикт.
- **Ключевые эндпоинты API:**
 - POST /applications: Принять новую заявку на скоринг.
 - GET /applications/{id}: Получить результат скоринга по заявке.

8. Loyalty Service

- **Назначение:**
Управляет всеми аспектами программ лояльности, в первую очередь — кешбэком.
- **Ключевые обязанности:**
 - Хранение Правил: Содержит правила начисления кешбэка (проценты, категории, лимиты, партнеры).
 - Расчет Кешбэка: Асинхронно обрабатывает события о покупках и рассчитывает сумму кешбэка.
 - Ведение Бонусного Счета: Учитывает накопленный, но еще не выплаченный кешбэк для каждого клиента.
 - Инициация Выплаты: По расписанию или по команде пользователя инициирует выплату накопленного кешбэка на реальный счет клиента.
- **Взаимодействие с другими сервисами:**
 - Синхронные вызовы (REST):
 - -> Вызывает (Calls): Payments Service для выполнения выплаты кешбэка.
 - <- Вызывается (Is Called By): Client API Gateway (для просмотра баланса кешбэка и выбора категорий).
 - Асинхронные взаимодействия (Kafka):
 - <- Слушает (Listens to): Исключительно асинхронный сервис. Его основная работа — слушать топик с событиями TransactionCompleted от Payments Service, чтобы начислять кешбэк.
- **База данных и хранимые сущности:**
 - Технология: PostgreSQL.
 - Ключевые таблицы:
 - CASHBACK_RULES: Правила и ставки кешбэка.
 - USER_CATEGORIES: Выбранные пользователем повышенные категории.
 - LOYALTY_ACCOUNTS: Бонусные счета клиентов с накопленными суммами.
- **Ключевые эндпоинты API:**
 - GET /cashback/balance: Получить текущий накопленный кешбэк.
 - POST /cashback/payout: Инициировать выплату кешбэка.
 - PUT /cashback/categories: Установить выбранные категории на следующий месяц.

9. Client Profile Service

- **Назначение:**
Является единственным источником правды для всей информации о клиенте, за исключением его финансовых продуктов. Центральный сервис для управления персональными данными (ПДн).
- **Ключевые обязанности:**
 - Управление ПДн: Хранит и предоставляет CRUD-операции для персональных данных клиента (ФИО, дата рождения, паспортные данные).
 - Управление Kontakтами: Хранит и управляет контактными данными (телефон, email, Telegram).
 - Управление Настройками: Хранит пользовательские настройки, например, выбранные каналы для уведомлений.
 - Интеграция с Keucloak: Взаимодействует с Keucloak для создания, блокировки и управления учетной записью клиента в системе аутентификации.

- Предоставление abs_client_id: Хранит связь между ID клиента в СДБО и его ID в Симуляторе АБС.
- **Взаимодействие с другими сервисами:**
 - Синхронные вызовы (REST):
 - -> Вызывает (Calls): Keycloak (через Admin API для управления пользователями).
 - <- Вызывается (Is Called By):
 - Client API Gateway: для отображения и редактирования профиля.
 - Operator API Gateway: для просмотра и управления данными клиента оператором.
 - Scoring Service: для получения анкетных данных при оценке кредитной заявки.
 - Products Service: для получения abs_client_id при создании нового продукта.
 - Асинхронные взаимодействия (Kafka):
 - -> Публикует (Publishes): События об изменении данных клиента, например, ClientProfileChanged, ClientPhoneNumberUpdated.
- **База данных и хранимые сущности:**
 - Технология: PostgreSQL.
 - Ключевые таблицы:
 - CLIENTS: Основная таблица с ПДн (id, keycloak_id, abs_client_id, first_name, last_name, passport_data).
 - CONTACTS: Контактные данные, связанные с клиентом (client_id, type (EMAIL/PHONE), value, is_verified).
 - USER_SETTINGS: Пользовательские настройки (client_id, setting_key, setting_value).
- **Ключевые эндпоинты API:**
 - GET /users/me: Получить профиль текущего пользователя.
 - PUT /users/me: Обновить профиль текущего пользователя.
 - GET /internal/users/{id}/abs-details: Внутренний эндпоинт для получения abs_client_id.
 - GET /admin/users: Получить список клиентов (для оператора).

10. Employee Management Service

- **Назначение:**
Изолированный сервис для управления жизненным циклом сотрудников банка (операторов, администраторов) и их правами доступа.
- **Ключевые обязанности:**
 - Управление Учетными Записями Сотрудников: Создание, редактирование, блокировка/разблокировка учетных записей сотрудников.
 - Управление Ролями: Назначение ролей (OPERATOR, ADMIN, SECURITY_OFFICER), которые определяют уровень доступа сотрудника в системе.
 - Интеграция с Keycloak: Взаимодействует с Keycloak для управления учетными записями и ролями сотрудников.
- **Взаимодействие с другими сервисами:**
 - Синхронные вызовы (REST):
 - -> Вызывает (Calls): Keycloak (через Admin API).
 - <- Вызывается (Is Called By): Исключительно Operator API Gateway для выполнения административных функций (например, администратор создает нового оператора).
 - Асинхронные взаимодействия (Kafka):
 - -> Публикует (Publishes): События для журнала аудита, например, EmployeeCreated, EmployeeRoleChanged.
- **База данных и хранимые сущности:**
 - Технология: PostgreSQL.
 - Ключевые таблицы:
 - EMPLOYEES: Профили сотрудников (id, keycloak_id, full_name, position).
 - ROLES: Справочник всех возможных ролей в системе.
 - EMPLOYEE_ROLES: Связующая таблица "многие-ко-многим" между сотрудниками и ролями.
- **Ключевые эндпоинты API:**
 - GET /admin/employees: Получить список всех сотрудников.
 - POST /admin/employees: Создать нового сотрудника.
 - PUT /admin/employees/{id}/roles: Изменить роли сотрудника.

11. History Service

- **Назначение:**
Создает и предоставляет клиентоориентированную историю операций. Его главная задача — быстро и удобно показать клиенту, что происходило с его деньгами.
- **Ключевые обязанности:**
 - Агрегация Событий: Слушает события из Kafka от разных сервисов (Payments, Products, Billing) и собирает их в единую ленту.
 - Денормализация Данных: Хранит данные в заранее подготовленном, денормализованном виде для максимально быстрых запросов на чтение. Не нужно делать JOIN нескольких таблиц в момент запроса.
 - Предоставление Истории: Отдает готовую историю операций с фильтрацией и пагинацией.

- Формирование Документов: Генерирует выписки по счетам и картам в форматах PDF/CSV на основе собранных данных.
- Предоставление Деталей Операции: Отдает полную, обогащенную информацию по конкретной операции.
- **Взаимодействие с другими сервисами:**
 - Синхронные вызовы (REST):
 - -> Вызывает (Calls): Практически никого. Работает в основном на чтение из своей базы.
 - <- Вызывается (Is Called By): Client API Gateway (для отображения ленты истории и выписок) и Operator API Gateway (для просмотра истории клиента оператором).
 - Асинхронные взаимодействия (Kafka):
 - <- Слушает (Listens to): Основной способ работы. Подписан на множество топиков, в первую очередь на TransactionCompleted от Payments Service, AccountOpened от Products Service и т.д.
- **База данных и хранимые сущности:**
 - Технология: PostgreSQL.
 - Ключевые таблицы:
 - HISTORY_EVENTS: Одна большая, "плоская" таблица, содержащая всю необходимую для отображения информацию: id, user_id, product_id, type (ПЕРЕВОД, ПОКУПКА), amount, currency, description, timestamp.
- **Ключевые эндпоинты API:**
 - GET /history: Получить ленту истории с пагинацией и фильтрами.
 - GET /history/{id}: Получить детали конкретной операции.
 - GET /statements?productId={id}&format=pdf: Сгенерировать и скачать выписку.

12. Audit Journal Service

- **Назначение:**
Служит централизованным, неизменяемым журналом аудита для записи всех критически важных для безопасности и анализа действий, происходящих в системе.
- **Ключевые обязанности:**
 - Запись Событий Безопасности: Асинхронно слушает события из Kafka и сохраняет их в защищенное хранилище.
 - Обогащение Событий: Добавляет к каждому событию метаданные: IP-адрес, user-agent, ID запроса для трассировки.
 - Неизменяемость (Append-only): Данные в журнал можно только добавлять, изменять или удалять их нельзя.
 - Предоставление API для Анализа: Предоставляет защищенный API для сотрудников службы безопасности или администраторов для поиска, фильтрации и анализа событий.
- **Взаимодействие с другими сервисами:**
 - Синхронные вызовы (REST):
 - -> Вызывает (Calls): Никого.
 - <- Вызывается (Is Called By): Исключительно Operator API Gateway для предоставления доступа к журналу сотрудникам с соответствующими правами.
 - Асинхронные взаимодействия (Kafka):
 - <- Слушает (Listens to): Основной способ работы. Подписан на все важные топики: TransactionCompleted, ClientProfileChanged, EmployeeRoleChanged, LoginAttemptFailed и т.д.
- **База данных и хранимые сущности:**
 - Технология: PostgreSQL или специализированная СУБД для логов, например, ClickHouse или Elasticsearch.
 - Ключевые таблицы:
 - AUDIT_LOGS: Одна большая таблица для всех событий: id, timestamp, service_name, event_type, user_id, target_entity_id, ip_address, full_payload_json.
- **Ключевые эндпоинты API:**
 - GET /admin/audit/logs: Поиск и фильтрация по журналу аудита.

13. Rates Service

- **Назначение:**
Является централизованным источником правды по валютным курсам. Изолирует всю сложность получения и расчета курсов от остальной системы.
- **Ключевые обязанности:**
 - Интеграция с Внешним Источником: По расписанию выполняет запросы к внешнему API (например, API ЦБ РФ) для получения официальных курсов валют.
 - Расчет Внутренних Курсов: Применяет бизнес-логику банка (маржу) для расчета внутренних курсов покупки и продажи.
 - Кэширование и Хранение: Хранит историю курсов и кэширует актуальные значения для быстрых ответов.
 - Предоставление API: Предоставляет внутренний REST API для других сервисов.
- **Взаимодействие с другими сервисами:**
 - Синхронные вызовы (REST):
 - -> Вызывает (Calls): Внешний API ЦБ РФ.

- <- Вызывается (Is Called By):
 - Client API Gateway: для отображения курсов на "витрине".
 - Payments Service: для получения "операционного" курса непосредственно перед конвертацией валюты.
 - Асинхронные взаимодействия (Kafka): Не участвует.
- **База данных и хранимые сущности:**
 - Технология: PostgreSQL для хранения истории, Redis для кэширования актуальных курсов.
 - Ключевые таблицы:
 - EXCHANGE_RATES_HISTORY: Таблица с историей официальных курсов, полученных от ЦБ.
 - INTERNAL_RATES_CONFIG: Конфигурация маржи банка для расчета внутренних курсов.
- **Ключевые эндпоинты API:**
 - GET /rates/public: Получить текущие "витринные" курсы покупки/продажи.
 - GET /rates/internal/current?from=USD&to=RUB: Получить точный курс для проведения финансовой операции.

14. Public Content Service

- **Назначение:**
Выполняет роль простой системы управления контентом (CMS) для всей публичной, справочной и редко меняющейся информации.
- **Ключевые обязанности:**
 - Хранение Контента: Управляет новостями, статьями, информацией о банке, адресами отделений и банкоматов.
 - Хранение Справочников: Хранит и предоставляет доступ к курируемым справочникам, таким как категории платежей и поставщики услуг (поиск по ИНН).
 - Предоставление API: Отдает контент по простому REST API.
- **Взаимодействие с другими сервисами:**
 - Синхронные вызовы (REST):
 - -> Вызывает (Calls): Никого.
 - <- Вызывается (Is Called By):
 - Client API Gateway: для отображения новостей, FAQ, категорий платежей, поиска поставщиков.
 - Operator API Gateway: для CRUD-операций по управлению контентом (добавление новости, редактирование справочника).
 - Асинхронные взаимодействия (Kafka): Не участвует.
- **База данных и хранимые сущности:**
 - Технология: PostgreSQL.
 - Ключевые таблицы:
 - NEWS_ARTICLES: Таблица с новостями.
 - FAQ_ITEMS: Таблица с вопросами и ответами.
 - PAYMENT_CATEGORIES: Справочник категорий платежей.
 - SERVICE_PROVIDERS: Справочник поставщиков услуг с их реквизитами.
- **Ключевые эндпоинты API:**
 - GET /news: Получить список новостей.
 - GET /payment-categories: Получить дерево категорий платежей.
 - GET /providers?search_inn={inn}: Найти поставщика по ИНН.
 - POST /admin/news: Создать новость (административный эндпоинт).

15. Notification Service

- **Назначение:**
Централизованный сервис, отвечающий за отправку всех исходящих уведомлений клиентам по различным каналам.
- **Ключевые обязанности:**
 - Реакция на События: Основная работа — слушать события в Kafka и инициировать отправку уведомлений.
 - Шаблонизация: Хранит шаблоны сообщений для разных событий ("Здравствуйте, {name}! Ваш перевод на сумму {amount} выполнен.").
 - Выбор Канала: На основе настроек пользователя (полученных из Client Profile Service) выбирает канал отправки: SMS, Push-уведомление или Email.
 - Интеграция со Шлюзами: Взаимодействует с внешними провайдерами для фактической отправки сообщений.
- **Взаимодействие с другими сервисами:**
 - Синхронные вызовы (REST):
 - -> Вызывает (Calls): Внешние шлюзы (SMS, Push, SMTP). Может запрашивать у Client Profile Service контактные данные.
 - <- Вызывается (Is Called By): Может иметь прямой REST API для срочных, не-событийных уведомлений (например, отправка кода подтверждения).
 - Асинхронные взаимодействия (Kafka):
 - <- Слушает (Listens to): Основной способ работы. Подписан на множество топиков: TransactionCompleted, CreditApplicationApproved, ClientProfileChanged и т.д.

- **База данных и хранимые сущности:**
 - Технология: PostgreSQL.
 - Ключевые таблицы:
 - MESSAGE_TEMPLATES: Шаблоны сообщений на разных языках.
 - NOTIFICATION_LOGS: Журнал всех отправленных (и не отправленных) уведомлений.
- **Ключевые эндпоинты API:**
 - POST /send-otp: Отправить одноразовый пароль (пример синхронного вызова).

16. Support Service

- **Назначение:**
Изолированный компонент, обеспечивающий работу клиентской поддержки через чат-бота и онлайн-чат с оператором.
- **Ключевые обязанности:**
 - Управление Диалогом: Ведет диалог с клиентом, используя логику чат-бота или проксируя сообщения к оператору.
 - API для Чата: Предоставляет API на основе WebSocket для обмена сообщениями в реальном времени.
 - Интеграция с Системой: Может выполнять аутентифицированные запросы от имени пользователя (с его разрешения) к другим сервисам для получения информации (например, "какой у меня баланс?").
 - Эскалация на Оператора: Обеспечивает механизм переключения диалога с бота на живого сотрудника.
- **Взаимодействие с другими сервисами:**
 - Синхронные вызовы (WebSocket/REST):
 - -> Вызывает (Calls): Может делать запросы через Client API Gateway для получения данных о клиенте (баланс, история), действуя как авторизованный клиент.
 - <- Вызывается (Is Called By): Клиентский Веб-Портал (для установления WebSocket-соединения).
 - Асинхронные взаимодействия (Kafka): Не участвует.
- **База данных и хранимые сущности:**
 - Технология: PostgreSQL.
 - Ключевые таблицы:
 - CHAT_SESSIONS: Сессии чатов.
 - CHAT_MESSAGES: История всех сообщений в чатах.
- **Ключевые эндпоинты API:**
 - WebSocket /chat: Основной эндпоинт для подключения к чату.